Лабораторна робота № 1

Реалізація штучного нейрона і перцептрона та дослідження функцій активації

Мета - закріпити математичну модель штучного нейрона, навчитися обчислювати зважену суму, застосовувати функції активації, реалізовувати перцептрон і досліджувати межі лінійно роздільної моделі у Google Colab.

Середовище виконання

Робота виконується у Google Colab. Основна мова — Python, основна бібліотека для нейронних мереж — PyTorch. Дозволяється безкоштовне середовище виконання CPU або GPU. Для відтворюваності блокнот повинен містити всі імпорти, фіксацію випадкових початкових значень, завантаження даних та послідовність виконання клітинок.

1. Відкрити https://colab.research.google.com/ і створити новий блокнот.
2. Перейменувати блокнот за схемою LR1_Прізвище.ipynb.
3. За потреби вибрати Середовище виконання → Змінити тип середовища → GPU.
4. Виконувати клітинки зверху вниз і зберегти результати обчислень та графіки.
5. Надати викладачеві посилання з правом перегляду або файл .ipynb.

Завдання

1. Створити блокнот у Google Colab та підключити NumPy, Matplotlib і PyTorch.

2. Для свого варіанта вручну обчислити зважену суму z та значення функцій Sigmoid і ReLU. Перевірити результат у Python.

3. Побудувати на одному рисунку графіки функцій Sigmoid і ReLU на проміжку від −5 до 5.

4. Сформувати таблицю істинності, навчити одношаровий перцептрон виконувати задану логічну операцію та подати таблицю підсумкових прогнозів.

Індивідуальні завдання за варіантами

Номер варіанта визначає викладач. Усі три операції є лінійно роздільними, тому одношаровий перцептрон повинен навчитися відтворювати відповідну таблицю істинності.

Варіант	Логічна операція	Дані для ручного розрахунку	Активації	η
1	AND: $y = x_1 \wedge x_2$	x=(1;1); w=(0,6;0,4); b=−0,5	Sigmoid, ReLU	0,10
2	OR: $y = x_1 \vee x_2$	x=(0;1); w=(0,7;0,3); b=−0,2	Sigmoid, ReLU	0,10
3	NAND: $y = \neg(x_1 \wedge x_2)$	x=(1;1); w=(−0,5;−0,7); b=0,8	Sigmoid, ReLU	0,10

Для навчання перцептрона використати повну таблицю істинності з чотирьох комбінацій входів: (0;0), (0;1), (1;0), (1;1). Максимальна кількість епох — 30.

Короткі теоретичні відомості

Штучний нейрон — це спрощена математична модель біологічного нейрона. Він отримує кілька вхідних значень, оцінює важливість кожного з них за допомогою ваг, додає зміщення та формує вихідне значення. У цій лабораторній роботі входи x_1 і x_2 набувають значень 0 або 1 і описують логічні умови.

Спочатку нейрон обчислює зважену суму $z = w_1x_1 + w_2x_2 + b$. Тут x_1 і x_2 — входи, w_1 і w_2 — ваги, а b — зміщення. Після цього до z застосовується функція активації: $y = f(z)$. Саме вона визначає, яке значення нейрон передасть далі.

Ваги та зміщення

Вага показує силу та напрям впливу входу. Додатна вага збільшує зважену суму, а від'ємна — зменшує її. Чим більший модуль ваги, тим помітніший вплив відповідного входу. Зміщення b дає змогу пересунути межу прийняття рішення і не прив'язувати її до початку координат.

Наприклад, для $x=(1;0)$, $w=(0{,}8;-0{,}4)$ і $b=-0{,}3$ маємо $z=0{,}8\cdot1+(-0{,}4)\cdot0-0{,}3=0{,}5$. Якщо використати порогове правило, за яким клас 1 обирається при $z\geq0$, нейрон поверне 1.

Функції активації

Функція активації перетворює зважену суму у вихід нейрона. У практичній частині достатньо дослідити Sigmoid і ReLU. Порогова функція використовується безпосередньо в алгоритмі перцептрона для отримання класу 0 або 1.

Функція	**Формула**	**Діапазон**	**Призначення**
Порогова	$f(z)=1$, якщо $z\geq0$; інакше 0	{0;1}	Класифікація перцептроном
Sigmoid	$f(z)=1/(1+e^{-z})$	(0;1)	Плавна оцінка, подібна до ймовірності
ReLU	$f(z)=\max(0;z)$	$[0;+\infty)$	Проста нелінійна активація

Sigmoid змінюється плавно: великі від'ємні значення наближаються до 0, нуль переходить у 0,5, а великі додатні значення — до 1. ReLU повертає 0 для від'ємних аргументів і саме значення z для додатних. Побудова графіків допомагає побачити цю різницю.

Перцептрон і його навчання

Перцептрон — найпростіший нейронний класифікатор. Для кожного навчального прикладу він обчислює прогноз $\hat{y}$. Потім прогноз порівнюється з правильною відповіддю y. Якщо відповіді не збігаються, ваги та зміщення коригуються так, щоб наступний прогноз став точнішим.

Помилка визначається як $e=y-\hat{y}$. Оновлення виконують за правилами $w\leftarrow w+\eta e x$ та $b\leftarrow b+\eta e$, де η — швидкість навчання. У цій роботі використовується $\eta=0{,}1$. Одна епоха означає один повний перегляд усіх чотирьох рядків таблиці істинності.

Послідовність навчання проста: задати початкові ваги; послідовно подати всі навчальні приклади; обчислити прогноз і помилку; за потреби оновити параметри; повторювати епохи, поки помилки не зникнуть або не буде досягнуто 30 епох.

Лінійна роздільність

Перцептрон може розв’язати задачу, якщо класи можна розділити однією прямою на площині. Операції AND, OR і NAND є лінійно роздільними, тому підходять для першої лабораторної роботи. Складніші операції, наприклад XOR, потребують мережі з прихованим шаром і розглядатимуться пізніше.

Робота у Google Colab

Google Colab дає змогу виконувати Python-код у браузері без локального встановлення програм. Для цієї роботи достатньо центрального процесора: обчислення дуже малі. NumPy використовується для масивів і скалярного добутку, Matplotlib — для графіків, а PyTorch підключається для знайомства з тензорами, що будуть потрібні в наступних роботах.

Клітинки блокнота потрібно виконувати зверху вниз. У звіті слід подати посилання на блокнот із правом перегляду, тому перед здаванням необхідно перевірити доступ і повторно виконати всі клітинки командою «Виконати все».

Приклад виконання

Наведений приклад показує мінімальну працездатну реалізацію. Його потрібно виконати у власному блокноті, перевірити результати, а потім змінити відповідно до індивідуального завдання.

```
import numpy as np
import torch
import matplotlib.pyplot as plt


# 1. Розрахунок виходу одного нейрона
x = np.array([1.0, 1.0])
w = np.array([0.6, 0.4])
b = -0.5
z = np.dot(w, x) + b
sigmoid_value = 1 / (1 + np.exp(-z))
relu_value = max(0, z)
print("z =", z, "Sigmoid =", sigmoid_value, "ReLU =", relu_value)


# 2. Графіки Sigmoid і ReLU
t = np.linspace(-5, 5, 400)
plt.plot(t, 1 / (1 + np.exp(-t)), label="Sigmoid")
```

```
plt.plot(t, np.maximum(0, t), label="ReLU")
plt.axhline(0, color="black", linewidth=0.5)
plt.axvline(0, color="black", linewidth=0.5)
plt.grid(alpha=0.25)
plt.legend()
plt.show()

# 3. Навчання перцептрона для AND
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([0, 0, 0, 1])
weights = np.zeros(2)
bias = 0.0
eta = 0.1

for epoch in range(30):
    errors = 0
    for features, target in zip(X, y):
        prediction = int(np.dot(weights, features) + bias >= 0)
        error = target - prediction
        weights += eta * error * features
        bias += eta * error
        errors += int(error != 0)
    if errors == 0:
        break

print("Ваги:", weights, "Зміщення:", bias)
for features in X:
    prediction = int(np.dot(weights, features) + bias >= 0)
    print(features.astype(int), "→", prediction)
```

Пояснення результату

У прикладі перцептрон коректно відтворює таблицю істинності AND. Під час зміни цільових значень на OR або NAND алгоритм також знаходить лінійну межу. Для XOR помилка не зникає, оскільки позитивні точки розташовані по діагоналі й не можуть бути відокремлені однією прямою.

Порядок виконання

1. Створити блокнот Google Colab, додати назву роботи, мету та номер варіанта.
2. Виконати ручний розрахунок нейрона і перевірити його програмно.
3. Побудувати графіки Sigmoid і ReLU.
4. Сформувати таблицю істинності та навчити перцептрон.
5. Перевірити прогнози для всіх чотирьох комбінацій входів і сформулювати висновок.

Вміст звіту з лабораторної роботи

1. Титульна сторінка, мета роботи та номер варіанта;
2. Посилання на блокнот Google Colab із правом перегляду;
3. Програмний код і графіки Sigmoid та ReLU;
4. Таблиця істинності та таблиця підсумкових прогнозів;
5. Отримані ваги, зміщення і короткий висновок.

Контрольні запитання

1. З яких елементів складається математична модель штучного нейрона?
2. Яку роль виконують ваги та зміщення?
3. Чим відрізняються функції Sigmoid і ReLU?
4. Що називають епохою навчання?
5. Як перцептрон змінює ваги після помилки?
6. Чому операції AND, OR і NAND можна реалізувати одним перцептроном?